SYSTEM DESIGN

How the L2 Finality Benchmark Works

Every component, what it does, what it hands to the next one, and the exact order things happen in.

PROJECT	50081 — L2 finality benchmark, Mitacs Globalink 2026
PURPOSE	Measure when an L2 transaction becomes final, at three levels of trust, and what it cost
APPROACH	Observational. Submit to public networks, read public evidence, host nothing
LANGUAGE	Python 3.10, web3.py 7.16
SCALE	~2,400 lines across 21 modules

1 · The one idea the whole system rests on

A transaction on a Layer 2 becomes "final" three separate times, and they can be hours apart.

LEVEL	FINAL WHEN	WHO TO TRUST
$\mathbf{t_1}$ full trust	the sequencer says yes	the rollup operator
$\mathbf{t_2}$ partial trust	the batch is posted to Ethereum	the operator, a bit less
$\mathbf{t_3}$ trustless	the batch is proven, or its challenge window closes	nobody

t_2 and t_3 are recorded on Ethereum, not on the rollup. The rollup operator must publish that evidence publicly for the system to work at all. So the benchmark does not need to run a rollup — it submits a transaction, then reads Ethereum to find out what happened to it.

That single fact is why the system looks the way it does: it is a *submitter* and a *reader*, and they are separate programs.

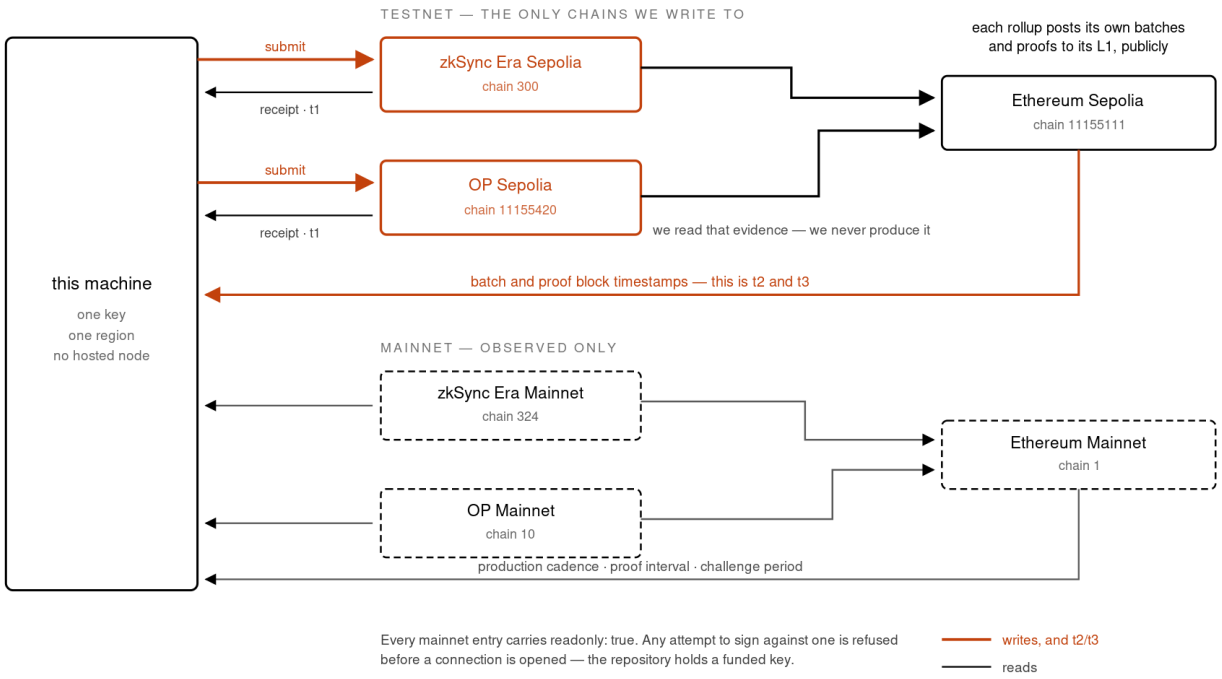


Figure 1. Six chains, two of which we write to. Everything else is read. The rollups publish their own batches and proofs to their L1 because their security depends on it; the study reads that evidence rather than producing any. Mainnet entries are flagged read-only and refuse a signature before a connection is even opened.

2 · The big picture

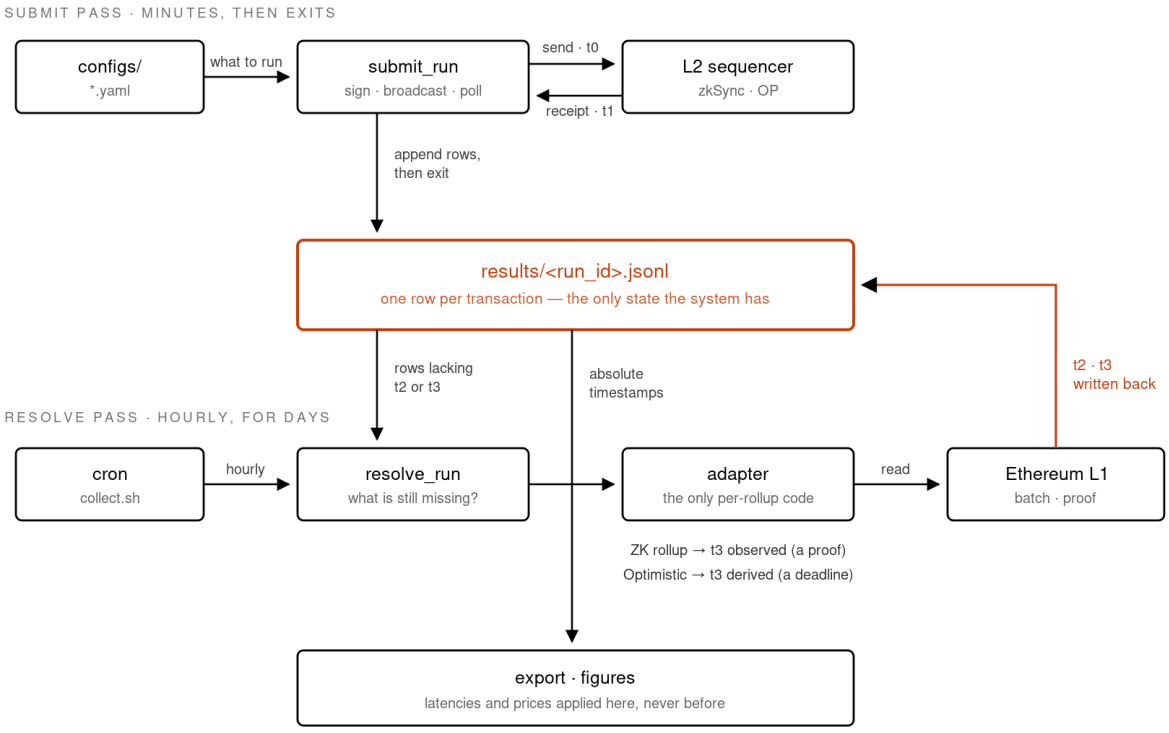


Figure 2. Submission and resolution are two separate programs joined only by a file. The submit pass finishes in minutes; the resolve pass runs hourly for as long as settlement takes — half an hour on a ZK rollup, seven days on an optimistic one. Neither knows the other exists.

Three layers, and the rule between them:

LAYER	FOLDER	RULE
Entry points	`bench/*.py`	Commands you run. No logic lives here, only orchestration
Core	`bench/core/`	Never knows which rollup it is talking to
Adapters	`bench/adapters/`	The only place per-rollup knowledge exists

3 · The submit pass, step by step

Command: `python -m bench.submit_run -n zksync_sepolia -w native_transfer -c 50`

1. **Read config.** Load `networks.yaml` for the endpoint and expected chain ID, and `workloads.yaml` for what to send. Nothing is hardcoded.
2. **Refuse read-only networks.** If the network is marked `readonly: true` (mainnet), stop here. The repository holds a funded key; this stops a typo spending real money.
3. **Connect and verify.** Open the endpoint and check the chain ID matches config. A mismatch aborts — an endpoint pointing at the wrong chain gives plausible results about the wrong system.
4. **Build the batch.** Make 50 transaction dictionaries. Selectors come from the committed ABI and arguments go through the ABI encoder; no calldata is ever assembled by hand.
5. **Record the L1 gas price.** Read Ethereum's gas price once, now. Comparing two runs made under very different L1 conditions is not a comparison, so the condition travels with the data.
6. **Assign nonces.** Fetch the account's transaction count *once*, then number the batch `base, base+1, base+2…` locally. Asking the node per transaction gives several of them the same number and all but one is rejected.
7. **Estimate gas.** Estimate on the first transaction and reuse it across an identical batch — fifty estimates would be fifty round trips for the same answer.
8. **Sign, then record t_0, then broadcast.** In that order, per transaction. t_0 is taken immediately before the send and after every local cost, so nothing local is blamed on the network.
9. **On failure, record the failure.** If the node refuses, write a row with `outcome: rejected`, the reason, and **hash left empty**. A fabricated hash would make every later measurement look plausible and mean nothing.
10. **Append rows to disk.** One JSON object per line into `results/<run_id>.jsonl`. Written now, before waiting for anything, so a crash loses nothing.
11. **Poll for receipts, round-robin.** Ask about every pending transaction in turn, sleep, repeat. Waiting out transaction 1 before first asking about transaction 50 would add that whole wait to transaction 50's latency.
12. **Record t_1 and the outcome.** Receipt status 1 is `success`, status 0 is `reverted`, no receipt in 120 s is `timeout` — a failure with a reason, never a very large latency.
13. **Check the nonce advanced.** Re-read the transaction count. If it moved less than expected, a transaction was dropped, which leaves a gap that stalls everything behind it.

14 **Rewrite the file and exit.** Rows now carry t_1. **The program stops here** — it never waits for settlement, which can take days.

4 · The resolve pass, step by step

Command: `python -m bench.resolve_run`. Safe to run any number of times; safe to kill at any moment.

1. **Read every run file.** Rows are normalised to the current schema on load, so a file written before a field existed still reads as a proper table.
2. **Decide what is outstanding.** A row needs work if it is missing t_2 or t_3, or lacks its batch identity. Rejected and timed-out rows are finished — they have no batch to wait for. This is recomputed from the file every pass, so nothing is remembered between runs.
3. **Pick the adapter.** By the network's `family` field: `zk` or `optimistic`. Raises if none exists rather than returning empty rows that look like "not settled yet".
4. **Ask the adapter where it settled.** Same call for every architecture: `settlement(w3_l2, w3_l1, network, tx_hash)`. What comes back is a `Settlement` — the L1 hashes, the batch, and how each was arrived at.
5. **Turn hashes into times.** For each L1 hash: fetch the transaction, read its block number, fetch the block, take its timestamp. Cached per hash, because a hundred of our rows usually share two L1 transactions.
6. **Refuse impossible values.** A settlement time earlier than t_0 is rejected and reported — a batch posted before our transaction existed cannot contain it.
7. **Write the file back atomically.** To a temporary file, then move it into place, so an interrupted pass cannot leave half a run file.

How the two adapters differ

	ZK ROLLUP (ZKSYNC)	OPTIMISTIC ROLLUP (OP)
t_2	Ask the rollup: receipt → batch number → batch details → commit hash. *Observed.*	No such mapping exists. Read the L2 block's L1 origin, then scan Ethereum forward for the batcher's next posting. *Estimated.*
t_3	The proof-verification transaction's block timestamp. *Observed* — a real event.	Find the dispute game covering the block, add the challenge period read from the portal. *Derived* — a deadline; nothing happens at that moment.
cost	Batch cost ÷ the batch's own transaction count, which the rollup exposes.	*Unavailable.* The count is not exposed, so there is no denominator. Reported as unavailable, never guessed.

WHY THE DIFFERENCE IS CARRIED IN THE DATA

Every timestamp is stored with a `_kind`: observed, estimated or derived. A proof verified in an Ethereum block and a challenge window that quietly expires are not the same kind of fact, and a table that prints them in the same column without saying so is misleading. The figures draw observed values solid and derived values dashed for the same reason.

5 · The data model

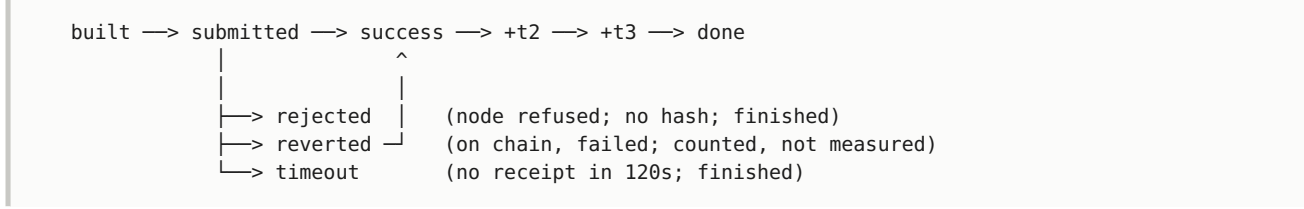
One row per transaction, one file per run, one JSON object per line. This file is the only state the system has.

```
{
  "run_id":        "zk_native_n50__rep2__20260811-0917",   which experiment cell
  "network":       "zksync_sepolia",
  "workload":      "native_transfer",
  "hash":          "0x32d4…",        the L2 transaction. Never invented.
  "nonce":         2,

  "t0":            1786349530.6,     we broadcast          <-- submit pass
  "t1":            1786349531.2,     we saw the receipt    <-- submit pass
  "t2":            1786351356,       batch posted to L1    <-- resolve pass
  "t3":            1786352028,       batch proven          <-- resolve pass

  "t2_kind":       "observed",       or "estimated"
  "t3_kind":       "observed",       or "derived"
  "l1_commit_tx":  "0xc090…",        anyone can open these
  "l1_prove_tx":   "0x27c4…",
  "batch":         21623,
  "batch_tx_count": 875,             everyone's, not just ours

  "outcome":       "success",        success | reverted | timeout | rejected
  "gas_used":      90916,            from the receipt, never an estimate
  "l1_gas_price_wei": 1095000000     L1 conditions at submission
}
```

Two rules about this shape

- **Only absolute timestamps are stored — never durations.** Every latency is computed as $t\_n - t_0$ at analysis time. An arithmetic mistake then costs a re-export, not a repeat of the experiment.
- **Prices are applied at analysis time too.** Rows hold gas and wei; dollars are derived. When the ETH rate turned out to be wrong by a factor of two, fixing every cost figure took one command.

The life of a row



6 · Configuration drives everything

FILE	CONTROLS
`networks.yaml`	Endpoints, chain IDs, explorers, L1 contract addresses, which chains are read-only
`workloads.yaml`	The two workloads, the recipient, the deployed token per network
`matrix.yaml`	Which combinations to run, how many repetitions, the minimum gap between them
`pricing.yaml`	The ETH/USD rate, when it was fetched and from where
`accounts.yaml`	The test address and how each network was funded
`.env`	The private key and any endpoint overrides. Never committed

`check_config.py` exists to prove none of these keys is decorative: it fails if any key is parsed and then never read. A setting you can change with no effect is the worst kind of bug in a benchmark, because the run still completes and the number does not move.

7 · What stops the system lying

GUARD	WHERE	STOPS
No fabricated hashes	`core/submit.py`	A failed send producing a fake ID that later code polls forever
Chain ID check	`core/networks.py`	Measuring the wrong network and never knowing
Stale endpoint check	`check_funds.py`	An RPC that answers correctly but serves week-old state
Read-only networks	`submit_run`, `deploy_token`	A typo spending real money on mainnet
Timestamp plausibility	`core/settle.py`	Settlement recorded as happening before submission
Invariant check	`check_data.py`	Impossible rows surviving into a figure
Dead config check	`check_config.py`	Settings that silently do nothing

> WHY SO MANY CHECKS
>
> Both data bugs this project has had produced *plausible numbers* rather than crashes — a settlement time nine seconds before submission, and 196 transactions given a finality timestamp 209 days before they were sent. Neither raised an error. Both were caught by a person noticing an odd median, which is luck rather than a control. The invariant check now runs on every hourly tick so the next one is caught by the machine.

8 · Unattended operation

One cron line: `17 * * * * bench/collect.sh`

1. **Submit one overdue cell.** `run_matrix --next` picks the cell with the fewest repetitions and refuses any run less than 60 minutes after that cell's last one. Five runs back to back would measure a single moment five times.
2. **Resolve whatever has settled.** Separate from submitting, because settlement takes half an hour on a ZK rollup and seven days on an optimistic one.

3 **Check the invariants.** Any violation is shouted into the log with an instruction not to trust figures built from those rows.

4 **Exit.** Nothing loops. A missed hour means the next tick picks up the most overdue cell; a machine asleep for six hours is six ticks behind, not broken.

9 · Adding another rollup

The design is judged by how little this takes.

1. Add an entry to `networks.yaml` — endpoint, chain ID, explorer, which L1 it settles on, its `family`.
2. If `family` is one that already exists, **stop. It works.**
3. Otherwise add one file to `adapters/` with a single method returning a `Settlement`.
4. Register it in the family map.

Nothing in `core/` changes, because nothing in `core/` knows what a rollup is. That is what let the same workload run unmodified across two architectures that disagree about what finality even means.

10 · Every command

COMMAND	DOES
`bench.create_wallet`	Make the test account
`bench.check_funds`	Balances, chain IDs, stale endpoints, what to do next
`bench.deploy_token`	Compile and deploy the ERC-20; `--dry-run` asks if the chain will take it
`bench.check_workloads`	Gas-estimate both workloads without sending
`bench.submit_run`	Send a batch, record t_0 and t_1
`bench.resolve_run`	Fill in t_2 and t_3
`bench.run_matrix`	The experiment plan; `--next`, `--paired`
`bench.observe_mainnet`	Read production cadence, send nothing
`bench.fetch_price`	Fetch and record the ETH rate with its sources
`bench.export`	Raw CSV, summary, assumptions, comparison table
`bench.analysis.figures`	The four figures, each with a CSV
`bench.analysis.reproducibility`	Day-to-day variation
`bench.check_data`	Are the rows possible?
`bench.check_config`	Does every setting do something?

11 · The shape in one paragraph

Configuration describes an experiment. A submitter turns it into real transactions and writes rows to a file, recording the moment of broadcast and the moment the sequencer acknowledged it, then exits within minutes. A resolver runs later and repeatedly, reading Ethereum to discover when each transaction's batch was posted and proven, and writing those times back into the same file. A checker asserts that the rows describe something possible. An exporter turns them into tables and figures, applying prices only at that

final step. The file is the state, the adapters are the only per-rollup code, and every number keeps a record of how it was obtained — observed, estimated, or derived.